

DE4W Capstone Project

Workbook 1: Training an Activity Recognition Model

Welcome

Welcome to the **DE4W Capstone Project**.

In this workbook, we build the training pipeline for a live activity recognition system.

The full system follows this pipeline:

```
Raw Sensor Data
→ Windowing
→ Feature Engineering
→ Machine Learning
→ Evaluation
→ Model Export
→ API
→ Dashboard
→ Live Prediction
```

Learning Objectives

After completing this workbook, you should be able to:

- load and inspect wearable sensor data
- understand smartphone accelerometer measurements
- convert continuous sensor streams into fixed-size windows
- extract statistical features from time-series windows
- train a Random Forest classifier
- compare random splits and subject-wise splits
- export a trained model for later deployment

Project Story

Imagine you joined a small digital health startup.

Your task is to build an application that recognizes whether a person is:

- walking
- sitting
- standing

using only the smartphone accelerometer.

The system should eventually work in real time.

Mission 1: Understand the Dataset

Goal

Before training a machine learning model, we first need to understand the data.

In this section we will:

- locate the raw WISDM phone accelerometer files
- load all participant files
- map activity codes to readable labels
- focus on walking, sitting and standing

Imports and Project Paths

joblib is used later to save the trained scikit-learn Model and reload it during Deployment.

Keeping paths relative to the project root makes the workbook easier to run on another computer.

```
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
```

```

        accuracy_score,
        classification_report,
        ConfusionMatrixDisplay,
    )
from sklearn.model_selection import train_test_split

PROJECT_ROOT = Path.cwd().parents[1] if Path.cwd().name == "workbooks" else Path.cwd()

DATA_DIR = PROJECT_ROOT / "data"
MODEL_DIR = PROJECT_ROOT / "models"

MODEL_DIR.mkdir(exist_ok=True)

print("Project root:", PROJECT_ROOT)
print("Data directory:", DATA_DIR)
print("Model directory:", MODEL_DIR)

```

Project root: /Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-Project/de4w-engineering-workbook
 Data directory: /Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-Project/de4w-engineering-workbook/data
 Model directory: /Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-Project/de4w-engineering-workbook/models

Activity Mapping

The WISDM raw files use single-letter activity codes.

```

ACTIVITY_MAP = {
    "A": "walking",
    "B": "jogging",
    "C": "stairs",
    "D": "sitting",
    "E": "standing",
    "F": "typing",
    "G": "brushing_teeth",
    "H": "eating_soup",
    "I": "eating_chips",
    "J": "eating_pasta",
    "K": "drinking",
    "L": "eating_sandwich",
}

```

```

    "M": "kicking",
    "O": "catch",
    "P": "dribbling",
    "Q": "writing",
    "R": "clapping",
    "S": "folding_clothes",
}

TARGET_ACTIVITIES = ["walking", "sitting", "standing"]

```

Load One WISDM File

Each raw file has the following structure:

```
subject_id, activity_code, timestamp, x, y, z;
```

```

def load_single_wisdm_file(file_path: Path) -> pd.DataFrame:
    """
    Load one WISDM phone accelerometer file.

    Parameters
    -----
    file_path:
        Path to one raw WISDM phone accelerometer text file.

    Returns
    -----
    pandas.DataFrame
        DataFrame with subject_id, activity_code, timestamp, x, y, z and activity.
    """
    df = pd.read_csv(
        file_path,
        header=None,
        names=["subject_id", "activity_code", "timestamp", "x", "y", "z"],
        sep=";",
    )

    df["z"] = (
        df["z"]
        .astype(str)
        .str.replace(";", "", regex=False)
    )

```

```

        .astype(float)
    )

    df["activity"] = df["activity_code"].map(ACTIVITY_MAP)

    return df

```

Load All Participant Files

```

def load_wisdm_phone_accel(
    data_dir: Path,
    target_activities: list[str] | None = None,
) -> pd.DataFrame:
    """
    Load all WISDM phone accelerometer files from a directory.
    """
    files = sorted(data_dir.glob("data*_accel_phone.txt"))

    if not files:
        raise FileNotFoundError(
            f"No WISDM phone accelerometer files found in {data_dir}."
        )

    dataframes = []

    for file_path in files:
        df_file = load_single_wisdm_file(file_path)

        if target_activities is not None:
            df_file = df_file[df_file["activity"].isin(target_activities)]

        dataframes.append(df_file)

    return pd.concat(dataframes, ignore_index=True)

```

```

df = load_wisdm_phone_accel(
    DATA_DIR,
    target_activities=TARGET_ACTIVITIES,
)

```

```
df.head()
```

	subject_id	activity_code	timestamp	x	y	z	activity
0	1600	A	252207666810782	-0.364761	8.793503	1.055084	walking
1	1600	A	252207717164786	-0.879730	9.768784	1.016998	walking
2	1600	A	252207767518790	2.001495	11.109070	2.619156	walking
3	1600	A	252207817872794	0.450623	12.651642	0.184555	walking
4	1600	A	252207868226798	-2.164352	13.928436	-4.422485	walking

```
df.shape
```

```
(814013, 7)
```

```
df["activity"].value_counts()
```

```
activity
walking    279817
standing   269604
sitting    264592
Name: count, dtype: int64
```

```
df["subject_id"].nunique()
```

```
51
```

Checkpoint 1

At this point, you should be able to answer:

1. What does one row in the raw WISDM file represent?
2. Why do we focus on only three activities first?
3. Why is it useful that the dataset contains subject identifiers?
4. Why should we inspect the class distribution before training a model?

Mission 2: Understand the Signal

Goal

We now inspect the accelerometer signal and compute the acceleration magnitude.

The magnitude combines all three axes:

$$m = \sqrt{x^2 + y^2 + z^2}$$

Add Magnitude

```
df["magnitude"] = np.sqrt(
    df["x"] ** 2 + df["y"] ** 2 + df["z"] ** 2
)

df.head()
```

	subject_id	activity_code	timestamp	x	y	z	activity	magnitude
0	1600	A	252207666810782	-0.364761	8.793503	1.055084	walking	8.864082
1	1600	A	252207717164786	-0.879730	9.768784	1.016998	walking	9.860900
2	1600	A	252207767518790	2.001495	11.109070	2.619156	walking	11.587812
3	1600	A	252207817872794	0.450623	12.651642	0.184555	walking	12.661010
4	1600	A	252207868226798	-2.164352	13.928436	-4.422485	walking	14.773088

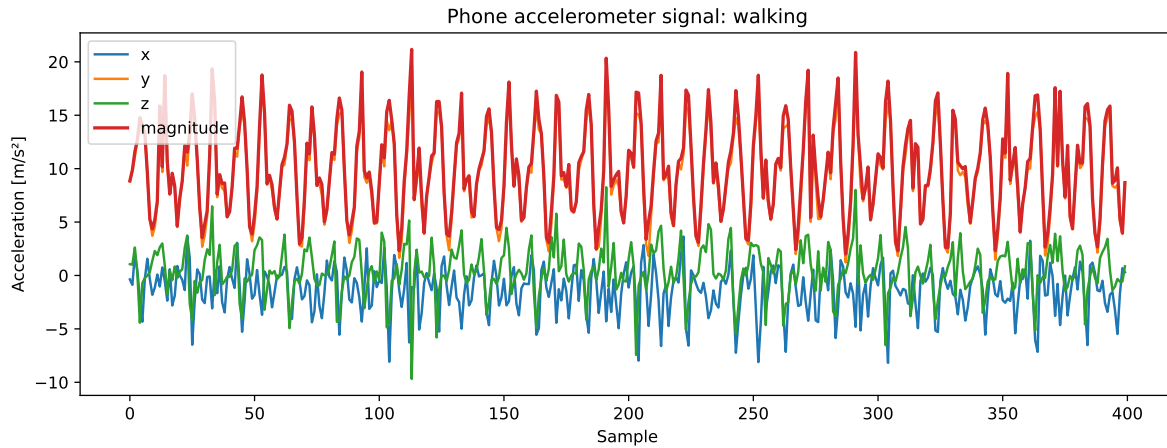
Plot One Activity

```
activity = "walking"

sample = (
    df[df["activity"] == activity]
    .head(400)
    .reset_index(drop=True)
)

plt.figure(figsize=(12, 4))
plt.plot(sample["x"], label="x")
```

```
plt.plot(sample["y"], label="y")
plt.plot(sample["z"], label="z")
plt.plot(sample["magnitude"], label="magnitude", linewidth=2)
plt.title(f"Phone accelerometer signal: {activity}")
plt.xlabel("Sample")
plt.ylabel("Acceleration [m/s2]")
plt.legend()
plt.show()
```

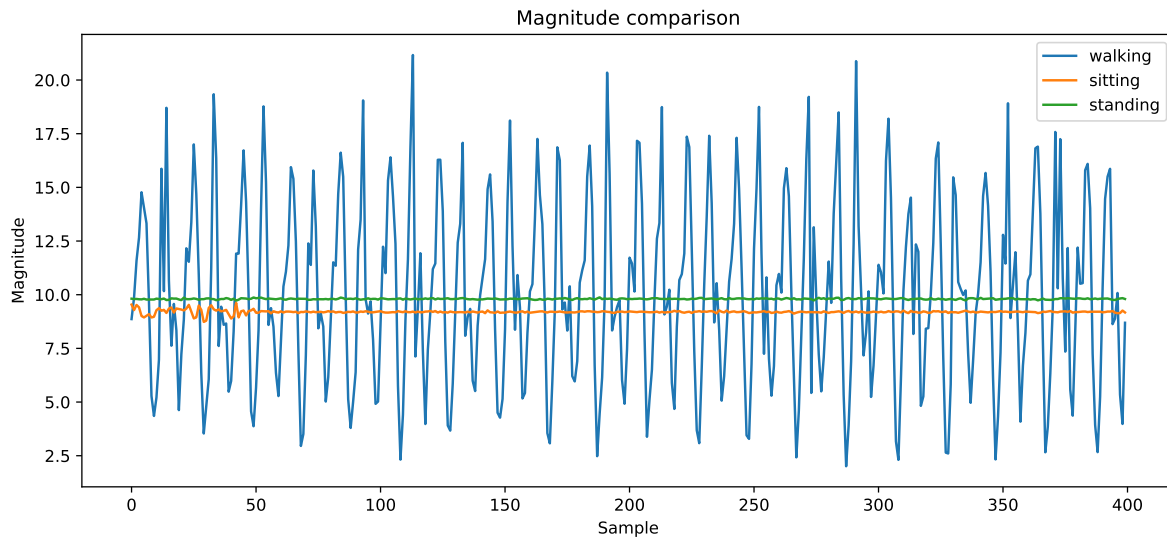


Compare Activities

```
plt.figure(figsize=(12, 5))

for activity in TARGET_ACTIVITIES:
    signal = (
        df[df["activity"] == activity]["magnitude"]
        .head(400)
        .reset_index(drop=True)
    )
    plt.plot(signal, label=activity)

plt.title("Magnitude comparison")
plt.xlabel("Sample")
plt.ylabel("Magnitude")
plt.legend()
plt.show()
```



Estimate Sampling Rate

```
subject = df["subject_id"].iloc[0]
activity = "walking"

sample = df[
    (df["subject_id"] == subject)
    & (df["activity"] == activity)
].sort_values("timestamp")

dt = sample["timestamp"].diff().dropna()
sampling_rate = 1e9 / dt.mean()

sampling_rate
```

```
np.float64(19.85939378331699)
```

Checkpoint 2

1. Why does the accelerometer not show zero acceleration when the phone is still?
2. What is the magnitude?
3. Why can magnitude be useful for activity recognition?
4. Why is sampling rate important for windowing?

Mission 3: Create Windows

Goal

We convert the continuous sensor stream into fixed-size windows.

In this project we use:

```
100 samples    5 seconds
```

because the sampling rate is approximately 20 Hz.

```
WINDOW_SIZE = 100
```

```
def create_windows(
    df: pd.DataFrame,
    window_size: int = WINDOW_SIZE,
) -> list[pd.DataFrame]:
    """
    Split a DataFrame into non-overlapping windows.
    """
    windows = []

    for start in range(0, len(df) - window_size, window_size):
        end = start + window_size
        windows.append(df.iloc[start:end])

    return windows
```

Test Windowing

```
walking_df = df[df["activity"] == "walking"].copy()

windows = create_windows(walking_df, window_size=WINDOW_SIZE)

len(windows)
```

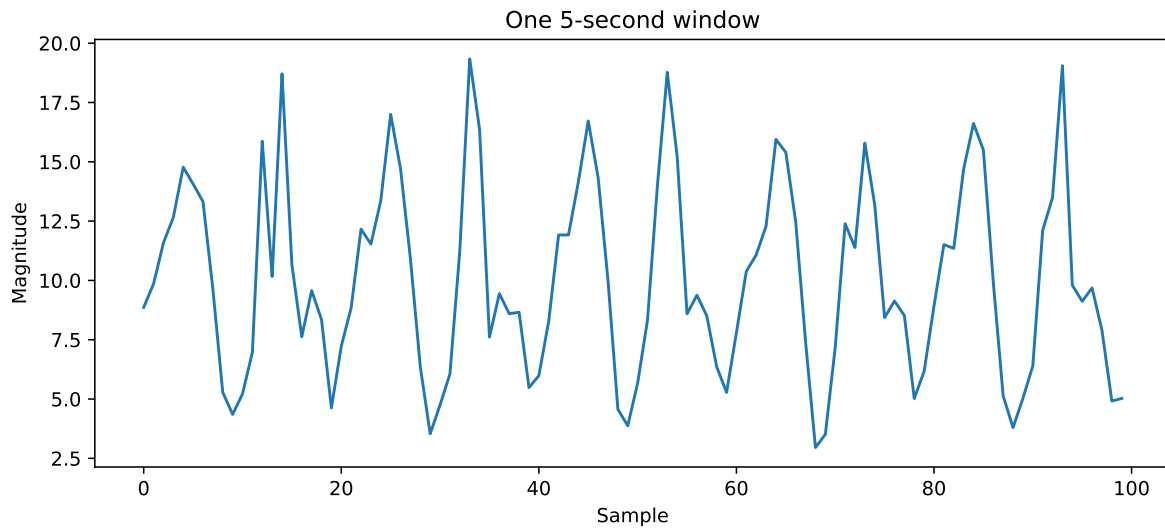
2798

```

window = windows[0]

plt.figure(figsize=(10, 4))
plt.plot(window["magnitude"])
plt.title("One 5-second window")
plt.xlabel("Sample")
plt.ylabel("Magnitude")
plt.show()

```



Checkpoint 3

1. Why do we need windows?
2. How many seconds are represented by 100 samples at 20 Hz?
3. What is the trade-off between small and large windows?
4. Why might live predictions not change immediately when the activity changes?

Mission 4: Extract Features

Goal

We transform each sensor window into a numerical feature vector.

For each signal:

```
x, y, z, magnitude
```

we compute:

```
mean, std, min, max, energy
```

This yields:

[4 × 5 = 20]

features.

```
def extract_features(window: pd.DataFrame) -> dict[str, float]:
    """
    Extract statistical features from one accelerometer window.
    """
    features = {}

    for axis in ["x", "y", "z", "magnitude"]:
        signal = window[axis]

        features[f"mean_{axis}"] = float(signal.mean())
        features[f"std_{axis}"] = float(signal.std())
        features[f"min_{axis}"] = float(signal.min())
        features[f"max_{axis}"] = float(signal.max())
        features[f"energy_{axis}"] = float(np.sum(signal ** 2))

    return features
```

Test Feature Extraction

```
features = extract_features(windows[0])

pd.Series(features)
```

mean_x	-1.000110
std_x	1.892580
min_x	-6.492264
max_x	3.651398
energy_x	454.626037

```
mean_y          9.619472
std_y           3.986348
min_y           2.684174
max_y           18.676529
energy_y        10826.630357
mean_z          0.340593
std_z           2.082760
min_z           -4.935730
max_z           6.472199
energy_z        441.051585
mean_magnitude  10.036322
std_magnitude   4.081904
min_magnitude   2.954282
max_magnitude   19.337960
energy_magnitude 11722.307978
dtype: float64
```

```
len(features)
```

20

Convert All Windows to Features

```
def windows_to_features(
    df: pd.DataFrame,
    window_size: int = WINDOW_SIZE,
) -> pd.DataFrame:
    """
    Convert raw sensor data into a feature table.
    """
    rows = []

    for activity in TARGET_ACTIVITIES:
        activity_df = df[df["activity"] == activity]

        for subject_id in sorted(activity_df["subject_id"].unique()):
            subject_df = activity_df[
                activity_df["subject_id"] == subject_id
            ].sort_values("timestamp")
```

```

windows = create_windows(subject_df, window_size=window_size)

for window in windows:
    feature_row = extract_features(window)
    feature_row["activity"] = activity
    feature_row["subject_id"] = subject_id
    rows.append(feature_row)

return pd.DataFrame(rows)

```

```

features_df = windows_to_features(df, window_size=WINDOW_SIZE)

features_df.head()

```

	mean_x	std_x	min_x	max_x	energy_x	mean_y	std_y	min_y	max_y	en
0	-1.000110	1.892580	-6.492264	3.651398	454.626037	9.619472	3.986348	2.684174	18.676529	10
1	-1.344509	2.100092	-8.082993	2.628891	617.398405	9.521806	4.066769	1.647736	18.787323	10
2	-1.513662	2.453901	-8.111908	3.641586	825.258859	9.580595	4.107310	1.222855	18.672226	10
3	-1.496280	2.116861	-8.181854	3.257019	667.514539	9.551068	4.078216	1.483902	18.526184	10
4	-1.644037	2.244849	-8.546417	5.301727	769.181090	9.362707	3.838999	1.363663	18.914047	10

```
features_df.shape
```

```
(8060, 22)
```

```
features_df["activity"].value_counts()
```

```

activity
walking      2772
standing     2670
sitting      2618
Name: count, dtype: int64

```

```
features_df.isna().sum().sum()
```

```
np.int64(0)
```

```
features_df.to_csv(
    DATA_DIR / "features_df.csv",
    index=False,
)
```

Checkpoint 4

1. Why do we extract features from windows?
2. Why might standard deviation help distinguish walking from standing?
3. What does energy measure?
4. Why must training and deployment use the same feature extraction function?

Mission 5: Train a Machine Learning Model

Goal

We train a Random Forest classifier to recognize walking, sitting and standing.

Prepare X and y

```
feature_columns = [
    column
    for column in features_df.columns
    if column not in ["activity", "subject_id"]
]

X = features_df[feature_columns]
y = features_df["activity"]

X.shape, y.shape
```

```
((8060, 20), (8060,))
```

Random Train/Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

```
rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
)

rf.fit(X_train, y_train)
```

`n_estimators` `n_estimators:` 200
int, default=100

The number of trees in the forest.

.. versionchanged:: 0.22
The default value of
“n_estimators” changed from
10 to 100
in 0.22.

random_state
random_state: int,
RandomState instance or
None, default=None

42

Controls both the
randomness of the
bootstrapping of the samples
used
when building trees (if
“bootstrap=True”) and the
sampling of the
features to consider when
looking for the best split at
each node
(if “max_features <
n_features”).
See :term:‘Glossary
<random_state>’ for details.
criterion criterion: {“gini”,
“entropy”, “log_loss”},
default=“gini”

‘gini’

The function to measure the
quality of a split. Supported
criteria are
“gini” for the Gini impurity
and “log_loss” and “entropy”
both for the
Shannon information gain,
see :ref:‘tree_mathemati-
cal_formulation’.
Note: This parameter is
tree-specific.

`max_depth` `max_depth: int, None`
`default=None`

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.
`min_samples_split` `min_samples_split: int or float, default=2`

2

The minimum number of samples required to split an internal node:

- If int, then consider ‘`min_samples_split`’ as the minimum number.
- If float, then ‘`min_samples_split`’ is a fraction and ‘`ceil(min_samples_split * n_samples)`’ are the minimum number of samples for each split.

.. versionchanged:: 0.18
Added float values for fractions.

`min_samples_leaf`
`min_samples_leaf`: int or
float, default=1

1

The minimum number of
samples required to be at a
leaf node.

A split point at any depth
will only be considered if it
leaves at
least “`min_samples_leaf`”
training samples in each of
the left and
right branches. This may
have the effect of smoothing
the model,
especially in regression.

- If int, then consider
‘`min_samples_leaf`’ as the
minimum number.
- If float, then
‘`min_samples_leaf`’ is a
fraction and
‘`ceil(min_samples_leaf *
n_samples)`’ are the
minimum
number of samples for each
node.

.. versionchanged:: 0.18
Added float values for
fractions.

<code>min_weight_fraction_leaf</code>	<code>0.0</code>
<code>min_weight_fraction_leaf:</code>	
float, default=0.0	

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

```
max_features max_features: 'sqrt'
{"sqrt", "log2", None}, int or
float, default="sqrt"
```

The number of features to consider when looking for the best split:

- If int, then consider ‘max_features’ features at each split.
- If float, then ‘max_features’ is a fraction and ‘max(1, int(max_features * n_features_in_))’ features are considered at each split.
- If "sqrt", then ‘max_features=sqrt(n_features)’.
- If "log2", then ‘max_features=log2(n_features)’.
- If None, then ‘max_features=n_features’.

.. versionchanged:: 1.1

The default of ‘max_features’ changed from “auto” to “sqrt”.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than “max_features” features.

<code>max_leaf_nodes</code> <code>max_leaf_nodes: int,</code> <code>default=None</code>	None
---	------

Grow trees with
“max_leaf_nodes“ in
best-first fashion.
Best nodes are defined as
relative reduction in impurity.
If None then unlimited
number of leaf nodes.

min_impurity_decrease 0.0
min_impurity_decrease:
float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following::

$$N_t / N * (\text{impurity} - N_{t_R} / N_t * \text{right_impurity} - N_{t_L} / N_t * \text{left_impurity})$$

where “N” is the total number of samples, “N_t” is the number of samples at the current node, “N_t_L” is the number of samples in the left child, and “N_t_R” is the number of samples in the right child.

“N”, “N_t”, “N_t_R” and “N_t_L” all refer to the weighted sum, if “sample_weight” is passed.

.. versionadded:: 0.19

<code>bootstrap</code>	<code>bootstrap: bool,</code>	<code>True</code>
	<code>default=True</code>	

Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

<code>oob_score</code>	<code>oob_score: bool</code>	<code>False</code>
	<code>or callable, default=False</code>	

Whether to use out-of-bag samples to estimate the generalization score. By default, :func:`~sklearn.metrics.accuracy\_score` is used. Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric. Only available if `'bootstrap=True'`.

For an illustration of out-of-bag (OOB) error estimation, see the example :ref:`sphx\_glr\_auto\_examples\_ensemble\_plot\_ensemble\_oob.py`.

<code>n_jobs</code>	<code>n_jobs: int,</code> <code>default=None</code>	None
The number of jobs to run in parallel. <code>:meth:'fit'</code>, <code>:meth:'predict'</code>, <code>:meth:'decision_path'</code> and <code>:meth:'apply'</code> are all parallelized over the trees. “None” means 1 unless in a <code>:obj:'joblib.parallel_backend'</code> context. “-1” means using all processors. See <code>:term:'Glossary <n_jobs>'</code> for more details.		
<code>verbose</code>	<code>verbose: int,</code> <code>default=0</code>	0
Controls the verbosity when fitting and predicting.		
<code>warm_start</code>	<code>warm_start:</code> <code>bool, default=False</code>	False
When set to “True”, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See <code>:term:'Glossary <warm_start>'</code> and <code>:ref:'tree_ensemble_warm_start'</code> for details.		

`class_weight` `class_weight`: None
{`"balanced"`,
`"balanced_subsample"`}, dict
or list of dicts, default=None

Weights associated with
classes in the form
“{`class_label`: `weight`}”.
If not given, all classes are
supposed to have weight one.
For
multi-output problems, a list
of dicts can be provided in
the same
order as the columns of `y`.

Note that for multioutput
(including multilabel) weights
should be
defined for each class of every
column in its own dict. For
example,
for four-class multilabel
classification weights should
be
[{0: 1, 1: 1}, {0: 1, 1: 5}, {0:
1, 1: 1}, {0: 1, 1: 1}] instead
of
[{1:1}, {2:5}, {3:1}, {4:1}].

The `"balanced"` mode uses
the values of `y` to
automatically adjust
weights inversely proportional
to class frequencies in the
input data
as `"n_samples / (n_classes *
np.bincount(y))"`

The `"balanced_subsample"`
mode is the same as
`"balanced"` except that
weights are computed based
on the bootstrap sample for
every tree
grown.

For multi-output, the weights
of each column of `y` will be
multiplied.

Note that these weights will
be multiplied with

`ccp_alpha` `ccp_alpha`: 0.0
non-negative float,
default=0.0

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “`ccp_alpha`” will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning’ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py’ for an example of such pruning.

.. versionadded:: 0.22

`max_samples` `max_samples`: None
int or float, default=None

If `bootstrap` is `True`, the
number of samples to draw
from `X`
to train each base estimator.

- If `None` (default), then draw
`'X.shape[0]'` samples
irrespective of
`'sample_weight'`.
- If `int`, then draw
`'max_samples'` samples.
- If `float`, then draw
`'max_samples * X.shape[0]'`
unweighted samples
or `'max_samples *
sample_weight.sum()'`
weighted samples.

.. versionadded:: 0.22

.. versionchanged:: 1.9
Float `'max_samples'` is
relative to
`'sample_weight.sum()'`
instead of
`'X.shape[0]'` for weighted
samples.

	<code>monotonic_cst</code> <code>monotonic_cst</code> : array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If <code>monotonic_cst</code> is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when 'n_classes > 2'), - multioutput classifications (i.e. when 'n_outputs_ > 1'). The constraints hold over the probability of the positive class. Read more in the :ref:'User Guide <monotonic_cst_gbdtd>'. .. versionadded:: 1.4	None
--	---	------

Name	Type	Value
------	------	-------

classes__ classes__: ndarray of shape (n_classes,) or a list of such arrays	ndarray[object](3,)	['sitting','standing','walking']
---	---------------------	----------------------------------

The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).

estimator__ estimator__: :class:~sklearn.tree.DecisionTreeClassifier	DecisionTreeClassifier	DecisionTreeClassifier()
--	------------------------	--------------------------

The child estimator template used to create the collection of fitted sub-estimators.

.. versionadded:: 1.2
 'base_estimator__' was renamed to 'estimator__'.
 estimators__ estimators__: list of DecisionTreeClassifier

The collection of fitted sub-estimators.

estimators__samples__ estimators__samples__: list of arrays	list
---	------

The subset of drawn samples (i.e., the in-bag samples) for each base estimator. Each subset is defined by an array of the indices selected.

.. versionadded:: 1.4

[Decision-TreeC...te=1608637542), Decision-TreeC...te=1273642419), Decision-TreeC...te=1935803228), Decision-TreeC...ate=787846414), ...]
 [array([2523, ..., dtype=int32), array([4782, ..., dtype=int32), array([5462, ..., dtype=int32), array([2848, ..., dtype=int32), ...]

<code>feature_importances_</code>	<code>ndarray[float64](20,)</code>	<code>[0.02,0.07,0.01,...,0.06,0.08,0.04]</code>
-----------------------------------	------------------------------------	--

`feature_importances_:`
 ndarray of shape
 (n_features,)

The impurity-based feature importances.
 The higher, the more important the feature.
 The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance.

Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:‘sklearn.inspection.permutation_importance’ as an alternative.

<code>feature_names_in_</code>	<code>ndarray[object](20,)</code>	<code>['mean_x','std_x','min_x',..., 'min_magnitude', 'energy_magnitude']</code>
--------------------------------	-----------------------------------	--

`feature_names_in_:` ndarray of shape (‘n_features_in’,)

Names of features seen during :term:‘fit’. Defined only when ‘X’ has feature names that are all strings.

.. versionadded:: 1.0

<code>n_classes_</code>	<code>n_classes_</code> : int or list	int	3
The number of classes (single output problem), or a list containing the number of classes for each output (multi-output problem).			
<code>n_features_in_</code>	<code>n_features_in_</code> : int	int	20
Number of features seen during :term:`fit`.			
<code>.. versionadded:: 0.24</code>			
<code>n_outputs_</code>	<code>n_outputs_</code> : int	int	1
The number of outputs when “fit” is performed.			

```
y_pred = rf.predict(X_test)

random_split_accuracy = accuracy_score(y_test, y_pred)

random_split_accuracy
```

```
0.989454094292804
```

```
print(classification_report(y_test, y_pred))
```

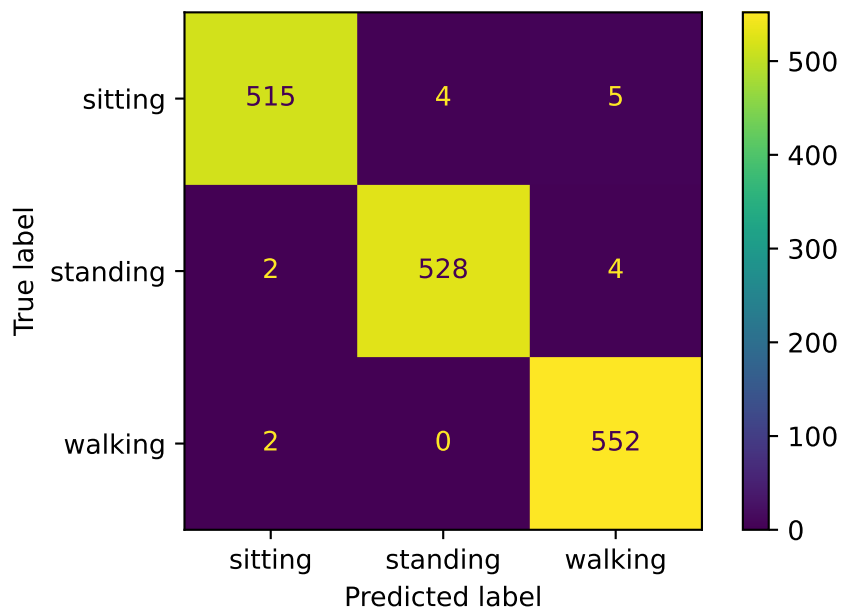
```

              precision    recall  f1-score   support

 sitting         0.99        0.98        0.99         524
 standing        0.99        0.99        0.99         534
 walking         0.98        1.00        0.99         554

 accuracy              0.99              0.99        1612
 macro avg          0.99        0.99        0.99        1612
 weighted avg       0.99        0.99        0.99        1612
```

```
ConfusionMatrixDisplay.from_estimator(
    rf,
    X_test,
    y_test,
)
plt.show()
```



Checkpoint 5

1. Why is Random Forest a good baseline?
2. What does accuracy measure?
3. What does the confusion matrix show?
4. Why might a random split be too optimistic?

Mission 6: Evaluate Generalization

Goal

We evaluate whether the model generalizes to unseen subjects.

```

subjects = features_df["subject_id"].unique()

train_subjects, test_subjects = train_test_split(
    subjects,
    test_size=0.2,
    random_state=42,
)

train_df = features_df[features_df["subject_id"].isin(train_subjects)].copy()
test_df = features_df[features_df["subject_id"].isin(test_subjects)].copy()

X_train_subject = train_df[feature_columns]
y_train_subject = train_df["activity"]

X_test_subject = test_df[feature_columns]
y_test_subject = test_df["activity"]

rf_subject = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
)

rf_subject.fit(X_train_subject, y_train_subject)

```

`n_estimators` `n_estimators:` 200
int, default=100

The number of trees in the forest.

.. versionchanged:: 0.22
The default value of “n_estimators” changed from 10 to 100 in 0.22.

random_state
random_state: int,
RandomState instance or
None, default=None

42

Controls both the
randomness of the
bootstrapping of the samples
used
when building trees (if
“bootstrap=True”) and the
sampling of the
features to consider when
looking for the best split at
each node
(if “max_features <
n_features”).
See :term:‘Glossary
<random_state>’ for details.
criterion criterion: {“gini”,
“entropy”, “log_loss”},
default=“gini”

‘gini’

The function to measure the
quality of a split. Supported
criteria are
“gini” for the Gini impurity
and “log_loss” and “entropy”
both for the
Shannon information gain,
see :ref:‘tree_mathemati-
cal_formulation’.
Note: This parameter is
tree-specific.

`max_depth` `max_depth: int, None`
`default=None`

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.
`min_samples_split` `min_samples_split: int or float, default=2`

2

The minimum number of samples required to split an internal node:

- If int, then consider ‘`min_samples_split`’ as the minimum number.
- If float, then ‘`min_samples_split`’ is a fraction and ‘`ceil(min_samples_split * n_samples)`’ are the minimum number of samples for each split.

.. versionchanged:: 0.18
Added float values for fractions.

`min_samples_leaf`
`min_samples_leaf`: int or
float, default=1

1

The minimum number of
samples required to be at a
leaf node.

A split point at any depth
will only be considered if it
leaves at
least “`min_samples_leaf`”
training samples in each of
the left and
right branches. This may
have the effect of smoothing
the model,
especially in regression.

- If int, then consider
‘`min_samples_leaf`’ as the
minimum number.
- If float, then
‘`min_samples_leaf`’ is a
fraction and
‘`ceil(min_samples_leaf *
n_samples)`’ are the
minimum
number of samples for each
node.

.. versionchanged:: 0.18
Added float values for
fractions.

<code>min_weight_fraction_leaf</code>	<code>0.0</code>
<code>min_weight_fraction_leaf:</code>	
float, default=0.0	

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

```
max_features max_features: 'sqrt'
{"sqrt", "log2", None}, int or
float, default="sqrt"
```

The number of features to consider when looking for the best split:

- If int, then consider ‘max_features’ features at each split.
- If float, then ‘max_features’ is a fraction and ‘max(1, int(max_features * n_features_in_))’ features are considered at each split.
- If "sqrt", then ‘max_features=sqrt(n_features)’.
- If "log2", then ‘max_features=log2(n_features)’.
- If None, then ‘max_features=n_features’.

.. versionchanged:: 1.1

The default of ‘max_features’ changed from “auto” to “sqrt”.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than “max_features” features.

<code>max_leaf_nodes</code> <code>max_leaf_nodes: int,</code> <code>default=None</code>	None
---	------

Grow trees with
“`max_leaf_nodes`” in
best-first fashion.
Best nodes are defined as
relative reduction in impurity.
If None then unlimited
number of leaf nodes.

min_impurity_decrease 0.0
min_impurity_decrease:
float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following::

$$N_t / N * (\text{impurity} - N_{t_R} / N_t * \text{right_impurity} - N_{t_L} / N_t * \text{left_impurity})$$

where “N” is the total number of samples, “N_t” is the number of samples at the current node, “N_t_L” is the number of samples in the left child, and “N_t_R” is the number of samples in the right child.

“N”, “N_t”, “N_t_R” and “N_t_L” all refer to the weighted sum, if “sample_weight” is passed.

.. versionadded:: 0.19

<code>bootstrap</code>	<code>bootstrap: bool,</code>	<code>True</code>
	<code>default=True</code>	

Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

<code>oob_score</code>	<code>oob_score: bool</code>	<code>False</code>
	<code>or callable, default=False</code>	

Whether to use out-of-bag samples to estimate the generalization score. By default, :func:`~sklearn.metrics.accuracy\_score` is used. Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric. Only available if `bootstrap=True`.

For an illustration of out-of-bag (OOB) error estimation, see the example :ref:`sphx\_glr\_auto\_examples\_ensemble\_plot\_ensemble\_oob.py`.

<code>n_jobs</code>	<code>n_jobs: int,</code> <code>default=None</code>	None
The number of jobs to run in parallel. <code>:meth:'fit'</code>, <code>:meth:'predict'</code>, <code>:meth:'decision_path'</code> and <code>:meth:'apply'</code> are all parallelized over the trees. “None” means 1 unless in a <code>:obj:'joblib.parallel_backend'</code> context. “-1” means using all processors. See <code>:term:'Glossary <n_jobs>'</code> for more details.		
<code>verbose</code>	<code>verbose: int,</code> <code>default=0</code>	0
Controls the verbosity when fitting and predicting.		
<code>warm_start</code>	<code>warm_start:</code> <code>bool, default=False</code>	False
When set to “True”, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See <code>:term:'Glossary <warm_start>'</code> and <code>:ref:'tree_ensemble_warm_start'</code> for details.		

`class_weight` `class_weight`: None
{`"balanced"`,
`"balanced_subsample"`}, dict
or list of dicts, default=None

Weights associated with
classes in the form
“{`class_label`: `weight`}”.
If not given, all classes are
supposed to have weight one.
For
multi-output problems, a list
of dicts can be provided in
the same
order as the columns of `y`.

Note that for multioutput
(including multilabel) weights
should be
defined for each class of every
column in its own dict. For
example,
for four-class multilabel
classification weights should
be
[{0: 1, 1: 1}, {0: 1, 1: 5}, {0:
1, 1: 1}, {0: 1, 1: 1}] instead
of
[{1:1}, {2:5}, {3:1}, {4:1}].

The `"balanced"` mode uses
the values of `y` to
automatically adjust
weights inversely proportional
to class frequencies in the
input data
as `"n_samples / (n_classes *
np.bincount(y))"`

The `"balanced_subsample"`
mode is the same as
`"balanced"` except that
weights are computed based
on the bootstrap sample for
every tree
grown.

44

For multi-output, the weights
of each column of `y` will be
multiplied.

Note that these weights will
be multiplied with

`ccp_alpha` `ccp_alpha`: 0.0
non-negative float,
default=0.0

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “`ccp_alpha`” will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning’ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py’ for an example of such pruning.

.. versionadded:: 0.22

`max_samples` `max_samples`: None
int or float, default=None

If bootstrap is True, the
number of samples to draw
from X
to train each base estimator.

- If None (default), then draw
'X.shape[0]' samples
irrespective of
'sample_weight'.
- If int, then draw
'max_samples' samples.
- If float, then draw
'max_samples * X.shape[0]'
unweighted samples
or 'max_samples *
sample_weight.sum()'
weighted samples.

.. versionadded:: 0.22

.. versionchanged:: 1.9
Float 'max_samples' is
relative to
'sample_weight.sum()'
instead of
'X.shape[0]' for weighted
samples.

	<code>monotonic_cst</code> <code>monotonic_cst</code>: array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If <code>monotonic_cst</code> is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when 'n_classes > 2'), - multioutput classifications (i.e. when 'n_outputs_ > 1'). The constraints hold over the probability of the positive class. Read more in the :ref:'User Guide <monotonic_cst_gbdtd>': .. versionadded:: 1.4	None
--	---	------

Name	Type	Value
------	------	-------

classes__ classes__: ndarray of shape (n_classes,) or a list of such arrays	ndarray[object](3,)	['sitting','standing','walking']
---	---------------------	----------------------------------

The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).

estimator__ estimator__: :class:'~sklearn.tree.DecisionTreeClassifier'	DecisionTreeClassifier	DecisionTreeClassifier()
--	------------------------	--------------------------

The child estimator template used to create the collection of fitted sub-estimators.

.. versionadded:: 1.2
 'base_estimator__' was renamed to 'estimator__'.
 estimators__ estimators__: list of DecisionTreeClassifier

The collection of fitted sub-estimators.

estimators__samples__ estimators__samples__: list of arrays	list
---	------

The subset of drawn samples (i.e., the in-bag samples) for each base estimator. Each subset is defined by an array of the indices selected.

.. versionadded:: 1.4

[Decision-
 TreeC...te=1608637542),
 Decision-
 TreeC...te=1273642419),
 Decision-
 TreeC...te=1935803228),
 Decision-
 TreeC...ate=787846414), ...]
 [array([2523, ...,
 dtype=int32), array([4782, ...,
 dtype=int32), array([5462, ...,
 dtype=int32), array([2848, ...,
 dtype=int32), ...]

<code>feature_importances_</code>	<code>ndarray[float64](20,)</code>	<code>[0.01,0.06,0.01,...,0.05,0.08,0.04]</code>
-----------------------------------	------------------------------------	--

`feature_importances_:`
 ndarray of shape
 (`n_features`,)

The impurity-based feature importances.
 The higher, the more important the feature.
 The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance.

Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See `:func:'sklearn.inspection.permutation_importance'` as an alternative.

<code>feature_names_in_</code>	<code>ndarray[object](20,)</code>	<code>['mean_x','std_x','min_x',..., 'min_magnitude', 'energy_magnitude']</code>
--------------------------------	-----------------------------------	--

`feature_names_in_:` ndarray of shape (`'n_features_in_'`,)

Names of features seen during `:term:'fit'`. Defined only when `'X'` has feature names that are all strings.

.. versionadded:: 1.0

n_classes_	n_classes_:	int	int	3
------------	-------------	-----	-----	---

or list

The number of classes (single output problem), or a list containing the number of classes for each output (multi-output problem).

n_features_in_	int	20
----------------	-----	----

n_features_in_: int

Number of features seen during :term:‘fit‘.

.. versionadded:: 0.24

n_outputs_	n_outputs_:	int	int	1
------------	-------------	-----	-----	---

The number of outputs when “fit” is performed.

```
y_pred_subject = rf_subject.predict(X_test_subject)

subject_split_accuracy = accuracy_score(y_test_subject, y_pred_subject)

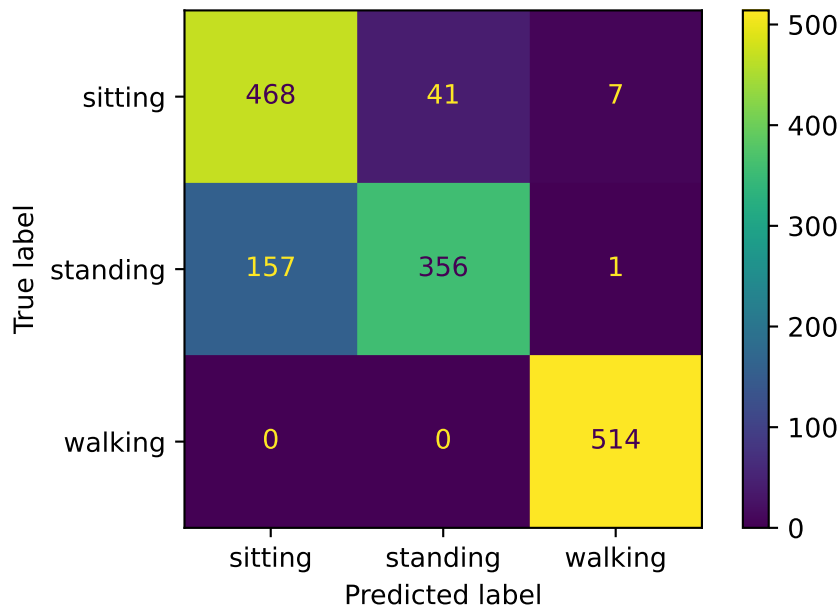
subject_split_accuracy
```

0.866580310880829

```
print(classification_report(y_test_subject, y_pred_subject))
```

	precision	recall	f1-score	support
sitting	0.75	0.91	0.82	516
standing	0.90	0.69	0.78	514
walking	0.98	1.00	0.99	514
accuracy			0.87	1544
macro avg	0.88	0.87	0.86	1544
weighted avg	0.88	0.87	0.86	1544

```
ConfusionMatrixDisplay.from_estimator(
    rf_subject,
    X_test_subject,
    y_test_subject,
)
plt.show()
```



Compare Both Evaluation Strategies

```
print("Random split accuracy:", random_split_accuracy)
print("Subject-wise split accuracy:", subject_split_accuracy)
```

Random split accuracy: 0.989454094292804
 Subject-wise split accuracy: 0.866580310880829

Checkpoint 6

1. Why is subject-wise evaluation more realistic?
2. Why is random split accuracy often higher?
3. Why are sitting and standing harder to distinguish than walking?
4. What does this tell us about deployment?

Mission 7: Export the Model

Goal

We export the trained model and metadata for later deployment.

Exporting with `joblib` turns trained Python objects into files. That is what lets the API and dashboard reuse the Model without retraining it.

```
joblib.dump(
    rf_subject,
    MODEL_DIR / "activity_model_random_forest.joblib",
)

joblib.dump(
    feature_columns,
    MODEL_DIR / "feature_columns.joblib",
)

activity_labels = sorted(features_df["activity"].unique())

joblib.dump(
    activity_labels,
    MODEL_DIR / "activity_labels.joblib",
)
```

```
['/Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-
Project/de4w-engineering-workbook/models/activity_labels.joblib']
```

Test Exported Model

```
loaded_model = joblib.load(MODEL_DIR / "activity_model_random_forest.joblib")
loaded_feature_columns = joblib.load(MODEL_DIR / "feature_columns.joblib")

sample_row = X_test_subject.iloc[[0]]

prediction = loaded_model.predict(sample_row[loaded_feature_columns])

prediction
```

```
array(['walking'], dtype=object)
```

```
pd.DataFrame(  
    loaded_model.predict_proba(sample_row[loaded_feature_columns]),  
    columns=loaded_model.classes_,  
)
```

	sitting	standing	walking
0	0.01	0.0	0.99

Final Checkpoint

You should now be able to explain the complete training pipeline:

```
Raw WISDM data  
→ Magnitude  
→ Windowing  
→ Feature extraction  
→ Feature table  
→ Random Forest  
→ Subject-wise evaluation  
→ Model export
```

Final Reflection

The next part of the project turns this trained model into a deployed system:

```
Model Export  
→ FastAPI  
→ Streamlit  
→ phyphox  
→ Live Prediction
```